Distributed Task Allocation Under Temporal Logic and Communication Constraints with Resilient Coordination

***Abstract*—This paper addresses resilient distributed task allocation for heterogeneous multi-robot systems under global syntactically co-safe linear temporal logic (sc-LTL) specifications with communication constraints. The task involves finite-region missions with temporal ordering, safety constraints, and collaborative subtasks requiring robot-type and cardinality consistency. In range-limited environments such as large-scale industrial facilities, communication links may become intermittently disconnected due to robot motion or failures, preventing task-state information (e.g., prerequisite completion and team composition) from propagating across the network. This leads to execution blocking of temporally dependent subtasks and failure of collaborative task formation, causing cascading delays in mission execution. To address this issue, we propose a resilient distributed execution framework that enables task progress under intermittent communication and dynamic robot removal. The framework constructs local executable plans from a global sc-LTL specification and restores execution feasibility by recovering missing prerequisite-state information and re-establishing feasible collaborative teams through communication-aware coordination. Simulations and physical experiments demonstrate reliable task completion and effective recovery of temporal and collaborative constraints under communication disruptions.**

***Index Terms*—Linear Temporal Logic, Multi-robot Systems, Task Allocation, Multi-robot Coordination.**

I. INTRODUCTION

Multi-robot systems (MRS) are increasingly important in applications such as patrol, surveillance, and logistics [1]–[3], because robot teams can provide higher efficiency, robustness, and flexibility than a single robot can. Multi-robot task allocation (MRTA) is therefore a fundamental problem in MRS. Existing task-allocation methods are generally divided into centralized and distributed approaches. Centralized methods compute assignments through a coordinator with global information [4]–[6], but they often suffer from high computational cost and single-point-of-failure issues, which limit scalability and robustness in dynamic environments. By contrast, distributed methods allow robots to make decisions based on local information and limited communication, including auction-based algorithms [7], [8], contract-net protocols [9], and rule-based methods [10]. These methods are especially attractive in communication-constrained scenarios such as security patrol and post-disaster search and rescue [11].

However, many practical multi-robot missions are not only communication-constrained, but also subject to temporal requirements such as ordering, persistence, and synchronization [12], which makes MRTA more challenging. To specify such requirements formally and precisely, recent studies have increasingly adopted formal languages [13], among which linear temporal logic (LTL) is one of the most widely used [14]. LTL provides a rigorous and unambiguous framework for task specification. For example, Li et al. [15], [16] studied heterogeneous MRTA under global LTL specifications with variable robot capabilities. Yu et al. [17] investigated fully distributed LTL-based multi-robot motion coordination using only local information, and employed online conflict detection and local replanning to preserve safety and specification feasibility. Despite this progress, sustaining global LTL missions in a distributed manner remains challenging under limited communication, because robots cannot maintain complete and timely team-state information during execution. This motivates the need for distributed task allocation methods that can sustain temporal-logic task under incomplete and delayed team-state information.

Under LTL constraints, existing task allocation and planning methods can generally be classified into top-down and bottom-up approaches. In top-down methods, a global LTL formula is used to specify the overall mission, and a centralized planner decomposes it into robot-level plans. A common strategy is to construct a product automaton from the nondeterministic Büchi automaton (NBA) derived from the global LTL formula and the discrete transition systems of the robots, and then solve the resulting graph-search problem [18], [19]. However, this approach suffers from severe state explosion in multi-robot settings and is difficult to adapt to dynamic execution scenarios involving robot addition or removal. Sampling-based methods [20], [21] improve upon the above methods by constructing generation trees from NBA samples rather than from the full product automaton. However, such methods typically require the candidate robots for each subtask to be specified in advance. Luo et al. [22] formulated temporal-logic task allocation as a mixed-integer programming problem; however, it is difficult to adapt online to practical execution contingencies such as robot failures. Chen et al. [23] proposed an incremental decision-tree-based method that supports online adaptation, but it relies on global information. Recognizing the parallel and concurrent nature of MRS, some studies decompose a global specification into multiple subtasks using automaton transition relations to increase parallelism. Schillinger et al. [24], [25] decomposed the global task into a set of parallel subtasks. However, this method does not explicitly handle collaborative subtasks. Based on these works, Liu et al. [26], [27] further improved efficiency by representing subtask temporal relations with posets. Recent studies also combine large language models with optimization-based schedulers, such as DEXTER-LLM [28]. Overall, although advanced top-down methods can derive task assignments from global temporal-logic specifications, handle collaborative subtasks, and in some cases support online adaptation, they still largely rely on centralized coordination and reliable global information. As a result, they are difficult to deploy with limited communication.

Bottom-up approaches, in contrast, usually assume that each robot is assigned a local temporal-logic task, and distributed coordination is then achieved through neighbor

communication and online replanning. Filippidis et al. [29] proposed a distributed planning framework under temporal logic constraints that enables robots to exchange information and verify conflicts online. Each robot can communicate only with neighbors within its communication range, and a dedicated "follower" robot is used to establish and maintain communication links when interaction is required. In [30], [31], each robot updates its knowledge based on local observations and propagates it through neighbor communication; with newly received knowledge, robots confirm and adjust their action paths online. However, such neighbor-only communication cannot guarantee that all robots required for a collaborative subtask can mutually communicate when needed. For scenarios with collaborative tasks, Tian et al. [32] proposed a distributed cooperative computation framework that decomposes task constraints via an interaction-edge product automaton, enabling robots to synthesize local trajectories in parallel using only neighborhood interaction information. Most existing work focuses on avoiding physical collisions while overlooking task conflicts. Chen et al. [33] addressed task-level conflict resolution under communication constraints by predicting the goals of robots beyond communication range. Compared with top-down approaches, bottom-up methods are generally more compatible with communication constraints. However, many of them assume that local temporal-logic tasks are preassigned, which limits their applicability when the task is specified only by a global formula.

This gap becomes significant in applications such as photovoltaic power station patrol and large-scale facility monitoring, where communication is restricted and task requirements are more naturally specified as a global LTL formula. In such settings, the challenge is not only to obtain an initial feasible allocation, but also to sustain task coordination during execution. Since robots cannot maintain a complete and timely team-state view, a subtask that is feasible at the planning stage may later become unrealizable because of dynamic changes in team composition, especially irreversible robot removals caused by failures. In particular, two major challenges arise during mission execution. First, a robot may be unable to verify whether the temporal prerequisites of its pending subtasks have been satisfied, which leads to task blocking. Second, a collaborative subtask may fail to start because the required robots do not arrive in time. Existing recovery strategies, including role reassignment, task reprioritization, and subtask reissuing [34]–[36], can alleviate these issues to some extent, but they often operate from fragmented local views. Consequently, failures due to the lack of global information may substantially increase the overall mission completion time.

Motivated by these challenges, this work develops a distributed task allocation framework with resilient task coordination. The framework combines distributed task allocation with a patrol mechanism for resolving blocked subtasks and a resilient coordination mechanism for collaborative subtasks. The main contributions are summarized as follows:

- A distributed task-allocation and coordination framework is proposed for heterogeneous robot teams under global sc-LTL specifications and limited communication. The framework derives executable local plans from a global specification and maintains task progress during execution using locally available and intermittently updated team-state information. Theoretical analysis establishes completeness of our approach under assumptions.
- A patrol mechanism is developed for temporally blocked subtasks. Instead of passively waiting for prerequisite information, idle robots actively gather information to release blocked subtasks.
- To strengthen task coordination for collaborative subtasks, two complementary mechanisms are designed: a leader-based coordination with timeout-triggered recruitment to reduce subtask failures, and a free-market-based reassignment to mitigate significant delays when subtask failures occur. Here, the free market is a designated rendezvous region where robots synchronize state information and recover failed collaborative subtasks.

Notations: Let $\mathbb{N}$ and $[N]$ denote the set of natural numbers and the index set $\{1, \ldots, N\}$, respectively. Given a set $\mathcal{A}$, let $|\mathcal{A}|$ and $2^{\mathcal{A}}$ denote the cardinality and power set of $\mathcal{A}$, respectively.

II. PRELIMINARIES

LTL formulas are constructed from atomic propositions $\mathcal{AP}$ using Boolean operators and temporal operators. Atomic propositions represent basic Boolean variables whose values are evaluated as either true or false. Details can be found in [14]. LTL formulas can be translated into NBA.

Definition 1 (Nondeterministic Büchi Automaton)**.** *A nondeterministic Büchi automaton (NBA) is a tuple* $\mathcal{B} = (\mathcal{S}, \mathcal{S}_0, \Sigma, \delta, \mathcal{F})$*, where* $\mathcal{S}$ *is a finite set of states,* $\mathcal{S}_0 \subseteq \mathcal{S}$ *is the set of initial states,* $\Sigma = 2^{\mathcal{AP}}$ *is the input alphabet,* $\delta : \mathcal{S} \times \Sigma \to 2^{\mathcal{S}}$ *is the transition function, and* $\mathcal{F} \subseteq \mathcal{S}$ *is the set of accepting states.*

Given an infinite word $\omega = \sigma_0\sigma_1\sigma_2 \ldots$ with $\sigma_i \in \Sigma$, a run $\rho = s_0 s_1 s_2 \ldots$ of the NBA $\mathcal{B}$ over ω is an infinite state sequence such that $s_0 \in S_0$ and $s_{i+1} \in \delta(s_i, \sigma_i)$ for all $i \geq 0$. The run ρ is accepting if it visits states in F infinitely often. Accepting runs can be represented in a prefix–suffix structure, where the prefix w_{pre} runs from an initial state to an accepting state, and the suffix w_{suf} forms a cycle containing at least one accepting state.

This paper focuses on a special class of LTL formulas called syntactically co-safe LTL (sc-LTL) [37], for which every satisfying execution has a finite good prefix. sc-LTL formulas restrict temporal operators to $\bigcirc$ (next), $\cup$ (until), and $\Diamond$ (eventually), and must be written in a positive normal form.

III. PROBLEM STATEMENT

A. Environment and Multi-Robot System

In a discrete workspace $\mathcal{W} \subseteq \mathbb{R}^2$, let $\mathcal{L} = \{l_1, l_2, \ldots, l_M\}$ be M pairwise disjoint target regions, i.e., $l_i \cap l_j = \emptyset$ for all $i \neq j$, where $i, j \in [M]$. Let $\mathcal{U}$ denote forbidden regions, and assume $l_i \cap \mathcal{U} = \emptyset$.

Given workspace $\mathcal{W}$, we consider a heterogeneous robot team $\mathcal{R} = \{r_1, r_2, \ldots, r_N\}$, where N is the number of robots.

The robots have n_t different types, where each robot belongs to exactly one type. Let $\mathcal{T} = [n_t]$ denote the type index set. For each $i \in \mathcal{T}$, let $\mathcal{R}_i \subseteq \mathcal{R}$ denote the set of robots of type i. Then $\mathcal{R} = \bigcup_{i=1}^{n_t} \mathcal{R}_i$ and $\mathcal{R}_i \cap \mathcal{R}_j = \emptyset$, $\forall i \neq j$. For any robot r_m, its velocity and position are denoted by v_m and x_m, respectively. Consider a time sequence $\{\tau_n = n\Delta t\}_{n\in\mathbb{N}}$ with sampling interval Δt. For convenience, for any quantity y, $y(\tau_n)$ denotes the value or state of y at time τ_n. In other words, the argument τ_n indicates the realization of the corresponding variable evaluated at τ_n.

Assuming communication range $D \in \mathbb{R}^+$, robots can only communicate with neighbors within this range. The neighbor set of robot r_m at time τ_n is defined as: $\mathcal{N}_m(\tau_n) = \{r_n \mid \|x_m(\tau_n) - x_n(\tau_n)\| \leq D\}$. The communication frequency is denoted by f_{com} in Hz.

B. Problem Description

To better describe the problem, we first introduce the atomic propositions. Let $a^m_{i,w}$ represent the action required for task completion, indicating that w robots of type i must execute this action at region $l_m \in \mathcal{L}$, where $i \in \mathcal{T}$, $w \in \mathbb{N}$, and $w \leq |\mathcal{R}_i|$. The following atomic propositions are then introduced: i) the atomic proposition p_m is true when any robot is located in region l_m, and false otherwise; ii) the atomic proposition $l^m_{i,w}$ is true iff action $a^m_{i,w}$ has been completed.

To facilitate task allocation under temporal logic constraints, a task model is introduced to explicitly capture the relationships among subtasks, actions, and workspace information. This abstraction provides a unified representation for subsequent task allocation.

Definition 2 (Task Model)**.** *The task model is defined as* $TM = (\mathcal{Q}, \mathcal{A}, \mathcal{W}, \text{LA}, \text{LC}, \text{LW})$*, where* $\mathcal{Q} := \{q_i\}$ *is the set of subtasks;* $\mathcal{A} := \{a^m_{i,w} \mid m \in [M],\ i \in [n_t],\ w \in [|\mathcal{R}_i|]\}$*. For notational simplicity, an action in* $\mathcal{A}$ *can be written as* a_j*, where* j *is the index.* $\mathcal{W}$ *is the workspace;* $\text{LA} : \mathcal{Q} \to 2^{\mathcal{A}}$ *associates each subtask with a set of actions, i.e.,* $\text{LA}(q_i) \subseteq \mathcal{A}$*;* $\text{LC} : \mathcal{A} \to \mathbb{R}_{\geq 0}$ *indicates the cost required to execute an action; and* $\text{LW} : \mathcal{A} \to \mathcal{W}$ *specifies the target region for an action.*

Given workspace $\mathcal{W}$ and NBA $\mathcal{B}$ corresponding to the LTL-specified global task φ, the plan for robot r_k is defined as a subtask-action-time sequence $\Pi_k = \Pi_{0,k}\Pi_{1,k}\cdots\Pi_{h_k,k}$, where each entry $\Pi_{i,k}$ is a subtask-action-time tuple associated with robot r_k, and h_k denotes the length of the plan. More specifically, $\Pi_{i,k} = (q_i, a_i, t_{i,k})$, where $q_i \in \mathcal{Q}$, $a_i \in \text{LA}(q_i)$, and $t_{i,k}$ denote the assigned subtask, the corresponding action, and the action start time of the i-th entry in the plan of robot r_k, respectively. In particular, the action of $\Pi_{0,k}$ is ε_a and the subtask is ε_q. Here, ε_q is the null subtask and ε_a is the null action. To simplify notation, we use $\Pi_{i,k}(q)$ to denote the subtask component of $\Pi_{i,k}$ and $\Pi_{i,k}(a)$ to denote its action component in the remainder of this paper. And for each action $a = a^m_{i,w} \in \mathcal{A}$, let $\text{type}(a) = i$ and $n_{\text{req}}(a) = w$ denote the required robot type and the required number of robots for executing a, respectively. For a subtask $q \in \Omega_\varphi$, its type-specific robot requirement is induced by its associated action set $L_A(q)$ and is defined as $n_{\text{req}}(q, j) = \sum_{a\in L_A(q):\, \text{type}(a)=j} n_{\text{req}}(a)$, where $j \in \mathcal{T}$. The total number of robots required by subtask q is then $n_{\text{req}}(q) = \sum_{j\in\mathcal{T}} n_{\text{req}}(q, j) = \sum_{a\in L_A(q)} n_{\text{req}}(a)$.

Example 1. *Consider a heterogeneous robot group* $\mathcal{R} = \{r_1, r_2, r_3, r_4\}$*, including two type-1 (e.g., inspection) and two type-2 (e.g., maintenance and manipulation) robots, performing distributed tasks in a large-scale industrial park. The target regions are* $\mathcal{L} = \{l_1, l_2, l_3, l_4\}$*. The task requires two type-1 robots to go to* l_1 *for a joint inspection of an abnormal facility status, and one type-2 robot must travel to a local service station at* l_2 *to perform an on-site handling operation. In addition, one type-1 and one type-2 robot must go to* l_3 *for a coordinated repair task. During this coordinated-repair period, no robot may pass through region* l_4*, which represents a hazardous area. Let* p_4 *denote the atomic proposition that any robot enters region* l_4*. The task can be described by the following sc-LTL formula.*

$$\varphi = \Diamond l^1_{1,2} \wedge \Diamond l^2_{2,1} \wedge \Diamond(l^3_{1,1} \wedge l^3_{2,1} \wedge \neg p_4) \tag{1}$$

Based on the above, the problem studied in this work is described as follows.

Problem 1. *Given a global sc-LTL task specification* φ*, workspace* $\mathcal{W}$*, and a team of heterogeneous robots* $\mathcal{R}$*, the goal is to develop a distributed task allocation and coordination strategy under range-limited communication. Specifically, the strategy should generate a plan* Π_k *for each robot* $r_k \in \mathcal{R}$ *and sustain task coordination during execution despite incomplete and delayed information. Let* $\Pi := \{\Pi_k\}_{r_k\in\mathcal{R}}$ *denote the collection of local plans. The objective is to generate* Π *such that it satisfies* φ*. The following assumptions are made.*

i) a robot can determine whether the task at a target region has been completed upon arrival;

ii) any robot within the broadcast range of a sender (another robot) will instantly receive its messages;

iii) the robot system contains sufficient robots.

IV. FRAMEWORK OVERVIEW

As shown in Fig. 1, the proposed framework consists of two parts: task preprocessing and distributed allocation with resilient task coordination.

In task preprocessing, the global sc-LTL specification is translated into a pruned automaton and decomposed into executable subtasks with temporal dependencies.

In the second part, robots allocate optional subtasks through consensus-based timetable algorithm (CBTA) and update their plans using local information. Since communication is limited, each robot maintains and broadcasts both its own task state and relayed peer information. For collaborative subtasks, the first robot arriving at the target region acts as the leader. If the required team does not form within the waiting time, timeout-triggered recruitment is activated. The leader first attempts local recovery and, if necessary, dispatches a recruiter to the free market; unsuccessful cases are then handled through free-market-based reassignment. In parallel, robots update their local plans through event-triggered replanning. When a robot cannot confirm that the temporal prerequisites of a pending

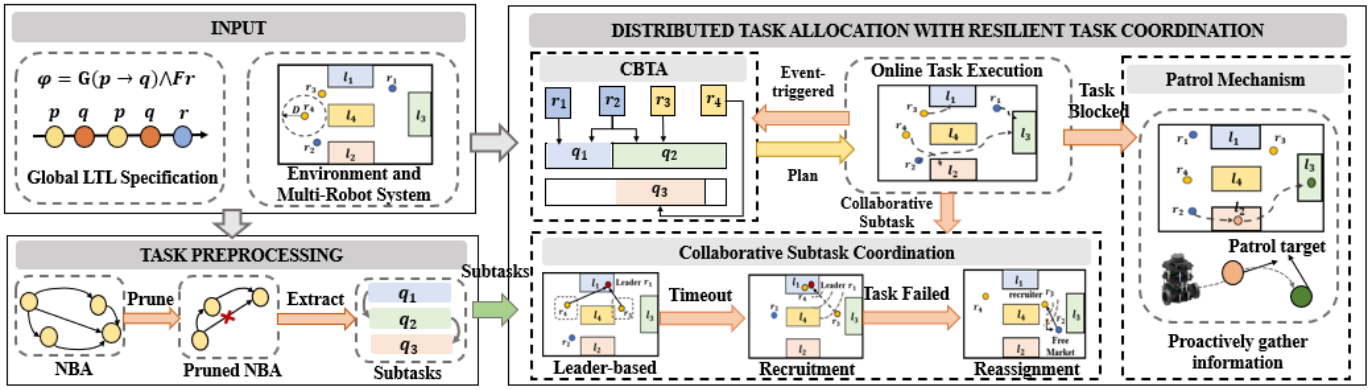

Fig. 1. Framework of the method. The framework consists of two stages: task preprocessing and distributed task allocation with resilient task coordination.

subtask have been satisfied, the patrol mechanism is activated to gather the missing information.

V. TASK PREPROCESSING

To address Problem 1, we preprocess φ. As shown in Fig. 1, preprocessing has two phases: i) NBA pruning; and ii) task decomposition on the pruned NBA to obtain the subtask set $\mathcal{Q}$ and the temporal relations among subtasks.

A. Pruning of NBA

The LTL formula is first converted into an NBA $\mathcal{B}$ and then pruned to remove redundancies, yielding a simplified automaton $\mathcal{B}^-$. Specifically, there are two types of pruning:

- *Removal of infeasible transitions:* removing transitions that cannot be realized, e.g., when the required number or types of robots exceed those available.
- *Elimination of decomposable transitions:* decomposable transitions are removed to improve task decomposition effectiveness [38].

A decomposable transition is defined as follows.

Definition 3 (Decomposable Transition)**.** *In NBA $\mathcal{B}$, a transition from state s_i to s_j is called decomposable if there exists another state s_k such that, for any $\sigma_{ik}, \sigma_{kj} \subseteq \Sigma$ satisfying $s_k \in \delta(s_i, \sigma_{ik})$ and $s_j \in \delta(s_k, \sigma_{kj})$, it holds that $s_j \in \delta(s_i, \sigma_{ik} \cup \sigma_{kj})$.*

Lemma 1. *For any feasible word w accepted by $\mathcal{B}$, there exists a corresponding word w' accepted by $\mathcal{B}^-$.*

Proof. Consider an accepted word $w = \cdots \sigma_n \cdots$ with accepting run $\rho = \cdots s_i s_j \cdots$ in $\mathcal{B}$, where $s_j \in \delta(s_i, \sigma_n)$. The first pruning step removes only infeasible transitions, so every feasible accepted word is preserved. For the second pruning step, consider two cases. For Case 1, if ρ contains no decomposable transitions, then all transitions of ρ remain in $\mathcal{B}^-$; hence ρ is also an accepting run in $\mathcal{B}^-$ and we can set $w' = w$. For Case 2, if a transition from s_i to s_j is removed as decomposable, there exists a state s_k and labels σ_{ik}, σ_{kj} such that $s_k \in \delta(s_i, \sigma_{ik})$, $s_j \in \delta(s_k, \sigma_{kj})$, and $\sigma_{ik} \cup \sigma_{kj} = \sigma_n$. Therefore, the transition $s_i \rightarrow s_j$ can be replaced by two transitions, $s_i \rightarrow s_k$ and $s_k \rightarrow s_j$. If both transitions $s_i \rightarrow s_k$ and $s_k \rightarrow s_j$ are non-decomposable, this reduces to Case 1. Otherwise, the same decomposition argument is applied recursively. Because a transition cannot be decomposed indefinitely into realizable finite expressions, this recursive process terminates. The final constructed run has no decomposable transitions, so it reduces to Case 1 and is accepted by $\mathcal{B}^-$. □

B. Task Decomposition

Following [39], $\mathcal{B}^-$ is decomposed into subtasks, and a relaxed partial order is extracted.A depth-first search on $\mathcal{B}^-$ is performed to generate accepting paths, and the path with the smallest number of subtasks is selected to obtain the corresponding task sequence. This selection follows the empirical rationale in [22]. Their results indicate that the first few returned posets often contain high-quality solutions. Starting from this ordered sequence, subtasks are iteratively swapped, and each new path is checked for acceptability. This process relaxes unnecessary temporal constraints and yields a logically consistent subtask set that supports parallel execution, together with a relaxed partial order.

Example 2. *For the task in Example 1, the subtask set corresponding to φ is given by $\Omega_\varphi = \{q_1, q_2, q_3\}$, where $q_1 = \{1, \{l^1_{1,2}\}, \{\}\}, q_2 = \{2, \{l^2_{2,1}\}, \{\}\}, q_3 = \{3, \{l^3_{1,1}, l^3_{2,1}\}, \{p_4\}\}$. Each subtask is represented by three entries: the first entry (e.g., 1, 2, 3) is the subtask index, the second set contains the actions to be executed, and the third set contains atomic propositions required on the self-loop.*

VI. DISTRIBUTED ALLOCATION WITH RESILIENT TASK COORDINATION

This section describes how feasible plans are generated under limited communication, and how these plans are executed .

A. Consensus-based Timetable Algorithm

To prevent robots from violating temporal logic constraints, the optional subtask set $\mathcal{O}_k(\tau_n)$ for robot r_k is defined.

Definition 4 (Optional Subtask Set)**.** *At time τ_n, the optional subtask set $\mathcal{O}_k(\tau_n)$ for robot r_k includes all subtasks $q_i \in \mathcal{Q}$*

that satisfy: i) the numbers or types of robots committed to q_i have not yet met the task requirements; ii) r_k is capable of q_i; iii) temporal constraints are satisfied; iv) q_i is not in the failed collaborative subtask set $\mathcal{F}_k$.

Note that robots committed to q_i include those traveling to, waiting in, or executing the subtask at the target region. To reduce failures in collaborative subtasks, a robot commits only when its currently available information indicates that enough planned participants, in both number and required types, are already allocated to that subtask.

Allocation within $\mathcal{O}_k(\tau_n)$ is performed using CBTA [7], which can efficiently allocate tasks by iterative timetable construction and consensus-reaching among robots. The cost of the current plan for robot r_k, i.e., the plan completion time, is defined below.

$$\mathrm{Cost}(\Pi_k(\tau_n)) = t_{h_k(\tau_n),k} - t_{0,k} + \mathrm{LC}(\Pi_{h_k(\tau_n),k}(a)) \quad (2)$$

Here, $h_k(\tau_n)$ is the current plan length of robot r_k at time τ_n; $t_{h_k(\tau_n),k}$ is the start time of the last action in $\Pi_k(\tau_n)$; $t_{0,k}$ is the start time of the plan; and $\mathrm{LC}(\Pi_{h_k(\tau_n),k}(a))$ is the execution cost of the last action. For each $q_i \in \mathcal{O}_k(\tau_n)$ and action $a_j \in \mathrm{LA}(q_i)$, candidate insertion positions after $\Pi_{0,k}(\tau_n)$ are evaluated by minimizing marginal cost, where $\Pi_{0,k}(\tau_n)$ is the initial tuple of the plan for robot r_k at time τ_n. The time component $t_{i,k} \in \Pi_{i,k}(\tau_n)$ is defined below.

$$t_{i,k} = \begin{cases} t_{0,k} + t_{\mathrm{left},k} + \dfrac{d_{i,k}}{v_k}, & \text{if } i = 1 \\ t_{i-1,k} + \mathrm{LC}(\Pi_{i-1,k}(a)) + \dfrac{d_{i-1,i}}{v_k}, & \text{otherwise} \end{cases} \quad (3)$$

where $t_{\mathrm{left},k}$ is the remaining time for the current subtask; for the case $i = 1$, $d_{i,k}$ is the distance from $x_k(\tau_n)$ to the target region of $\Pi_{i,k}(q)$; and $d_{i-1,i}$ is the distance between the target regions of $\Pi_{i-1,k}(q)$ and $\Pi_{i,k}(q)$. To enforce robot-quantity requirements, Eq. (2) is augmented with a penalty $M \sum_{i \in [h_k(\tau_n)]} n_{\mathrm{req}}(\Pi_{i,k}(q))$, where $n_{\mathrm{req}}(\Pi_{i,k}(q))$ is the number of robots required to accomplish subtask $\Pi_{i,k}(q)$ and M is a large constant.

Assume that $\mathcal{P}_k(\tau_n)$ is the set of plans received by robot r_k at time τ_n. Task-allocation conflicts are then resolved based on these shared plans as follows. For each candidate subtask $q_i \in \mathcal{O}_k(\tau_n)$ and each action $a_j \in \mathrm{LA}(q_i)$, robot r_k first counts the number of robots currently assigned to execute a_j according to $\mathcal{P}_k(\tau_n)$. Let $n^k_{\mathrm{req}}(a_j)$ denote the number of robots that r_k estimates to be required for a_j. If the number of assigned robots exceeds $n^k_{\mathrm{req}}(a_j)$, an over-assignment conflict is detected. In this case, only the first $n^k_{\mathrm{req}}(a_j)$ robots with the earliest expected action start times are retained. Otherwise, no task-allocation conflict is identified, and all currently assigned robots are retained. See [7] for algorithmic details.

Lemma 2. *Under the assumptions in Sec.III-B and the given local information, CBTA-based allocation satisfies the robot-type requirements, robot-number requirements, and the temporal-logic constraints.*

Proof. Following [39], the subtasks and their partial order are consistent with the original sc-LTL specification. It remains to show that the CBTA-based allocation does not violate these extracted subtask relations or the robot participation requirements. According to Definition 4, robot r_k can insert subtasks only from its optional-subtask set $\mathcal{O}_k(\tau_n)$. Therefore, any inserted subtask must satisfy the following conditions: its temporal prerequisites are satisfied according to the local task-state information available to r_k; robot r_k is capable of executing the corresponding action; and the subtask is not contained in the failed collaborative-subtask set maintained by r_k. Hence, no subtask that violates the extracted temporal dependencies is inserted into the local plan. For each candidate action $a_j \in \mathrm{LA}(q_i)$, robot r_k checks the number of robots currently committed to a_j using the received plan set $\mathcal{P}_k(\tau_n)$. If over-assignment occurs, only the first $n_{\mathrm{req}}(a_j)$ robots with the earliest expected action-start times are retained. Thus, a confirmed commitment never exceeds the required number of robots. Meanwhile, a collaborative subtask is allowed to proceed only when the required numbers and types of robots are available according to the local information. Therefore, the generated local plan is feasible with respect to the robot-type and robot-number requirements under the information available to r_k, while preserving the temporal-logic consistency induced by the extracted relaxed partial order. □

B. Free Market

Under range-limited communication, local information may be insufficient for idle robots or collaborative-subtask leaders to find suitable type-compatible robots. Therefore, we introduce the free market as a rendezvous mechanism for state synchronization and failed collaborative-subtask reassignment. The free market is a rendezvous region used for global information synchronization and failed collaborative subtask reassignment. A robot moves toward the free market in one of three situations: it has no executable subtask, unless all subtasks have already been completed; its assigned optional subtask cannot proceed because of failure or missing collaborators; or it is explicitly dispatched by a leader as a recruiter.

The location of the free market should be close to subtasks that are more likely to require coordination or to affect subsequent execution. Therefore, the free-market location is set as a weighted center of subtask regions. For each subtask $q_i \in \mathcal{Q}$, two quantities are used: the required robot count $n_{\mathrm{req}}(q_i)$ and the number of successor subtasks $N_{\mathrm{suc},i}$. Their normalized forms $\nu_{i,1}$ and $\nu_{i,2}$ are defined as

$$\nu_{i,1} = \frac{n_{\mathrm{req}}(q_i)}{\sum_{q_j \in \mathcal{Q}} n_{\mathrm{req}}(q_j)}, \quad \nu_{i,2} = \frac{N_{\mathrm{suc},i}}{\sum_{q_j \in \mathcal{Q}} N_{\mathrm{suc},j}}. \quad (4)$$

The entropy weight w_k of indicator $k \in \{1, 2\}$ is computed by

$$e_k = -\frac{1}{\ln |\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} \nu_{i,k} \ln \nu_{i,k}, \quad (5)$$

$$w_k = \frac{1 - e_k}{\sum_{m=1}^{2} (1 - e_m)}. \quad (6)$$

Here, $|\mathcal{Q}|$ is the number of subtasks. The normalized subtask weight $\bar{s}_i$ is

$$\bar{s}_i = \frac{w_1 \nu_{i,1} + w_2 \nu_{i,2}}{\sum_{q_j \in \mathcal{Q}} (w_1 \nu_{j,1} + w_2 \nu_{j,2})}, \quad (7)$$

and the nominal free-market center is

$$x_{\text{free}} = \sum_{q_i \in \mathcal{Q}} \bar{s}_i x_i. \tag{8}$$

If x_{free} lies in a forbidden region or outside the workspace, it is projected to the nearest feasible point, which is still denoted by x_{free} for notational simplicity.

The free market is not modeled as a predefined physical region with fixed size and shape. Instead, it is a communication-defined rendezvous region with the fixed center x_{free}. Its actual extent is formed by robots that are admitted to the current free-market robot set. Let $\mathcal{R}_f(\tau_n)$ denote the set of robots currently admitted to the free market at time τ_n. A robot is regarded as having arrived at the free market when it reaches a feasible point closest to x_{free} and lies within the communication range of at least one robot already in free market if such a robot exists. Therefore, the free-market communication graph is connected. Robots broadcast $\mathcal{R}_f$, along with the corresponding timestamp. Upon receiving this information, each robot updates its locally maintained free-market ID set $\mathcal{R}_f$ using the newest timestamp.

Algorithm 1 Timeout-Triggered Recruitment

1: **Input:** collaborative subtask q_c, arrived-robot set $\mathcal{R}_{\text{arr}}$, leader state-information set $\mathcal{S}_l(\tau_n)$, free-market location x_{free}, free-market robot set $\mathcal{R}_f$, reachable robot set $\mathcal{R}_b$
2: **if** $|\mathcal{R}_{\text{arr}}| < n_{\text{req}}(q_c)$ **then**
3: Construct $\mathcal{R}^l_{\text{fit}}$ from robots in $\mathcal{R}_b$ that are available and type-compatible with the missing roles of q_c.
4: **if** $|\mathcal{R}_{\text{arr}} \cup \mathcal{R}^l_{\text{fit}}| \geq n_{\text{req}}(q_c)$ **then**
5: Recruit the required robots from $\mathcal{R}^l_{\text{fit}}$.
6: Update T_{wait} via Eq. (9).
7: **if** the recruited robots arrive within the updated T_{wait} **then**
8: Leader r_l broadcasts a start signal for subtask q_c.
9: **else**
10: Leader r_l announces failure of subtask q_c.
11: **end if**
12: **end if**
13: Construct $\mathcal{R}^f_{\text{fit}}$ from robots in $\mathcal{R}_f$ that are type-compatible with the remaining missing roles of q_c.
14: **if** $|\mathcal{R}_{\text{arr}} \cup \mathcal{R}^l_{\text{fit}} \cup \mathcal{R}^f_{\text{fit}}| < n_{\text{req}}(q_c)$ **then**
15: Leader r_l announces failure of subtask q_c.
16: **end if**
17: Extract $\{x_p(\tau_n) \mid r_p \in \mathcal{R}_{\text{arr}}\}$ from $\mathcal{S}_l(\tau_n)$.
18: $r_p \leftarrow \arg\min_{r_p \in \mathcal{R}_{\text{arr}}} d(x_p(\tau_n), x_{\text{free}})$
19: Recruit robots in $\mathcal{R}^l_{\text{fit}}$ and dispatch r_p to the free market.
20: Compute timeout T^p_{leave} via Eq. (11).
21: **if** recruitment succeeds **then**
22: Leader r_l broadcasts a start signal for subtask q_c.
23: **else if** waiting time $t \geq T^p_{\text{leave}}$ or recruitment fails **then**
24: Leader r_l announces failure of subtask q_c.
25: **end if**
26: **end if**

C. Collaborative Subtask Coordination

1) Leader-based Collaborative Subtask Coordination: For collaborative subtasks, robots adopt a leader-based coordination strategy. One robot serves as a temporary leader to collect arrival information, synchronize decisions, and issue recruitment actions. Compared with fully leaderless coordination, this design reduces negotiation overhead and decision inconsistency, thereby improving coordination efficiency and execution robustness.

Upon arriving at a subtask region, robot r_k broadcasts an arrival signal $e_k = \{t^k_e, q_k, a_k\}$, where t^k_e is the arrival timestamp at the task region, and q_k and a_k denote the current subtask and the action performed by r_k, respectively. The robot that arrives first at the task region is selected as leader r_l to prevent conflicts caused by asynchronous arrivals. Let q_c denote the collaborative subtask currently coordinated by leader r_l. If the number of robots that arrive within the collaboration waiting time T_{wait} satisfies the requirement of q_c, the leader broadcasts a start signal and the robots begin executing the collaborative subtask. Likewise, after timeout-triggered recruitment, execution starts only after the recruited robots have arrived and the participation requirement of q_c is satisfied. Otherwise, once the waiting time exceeds T_{wait}, timeout-triggered recruitment is initiated.

The collaboration waiting time T_{wait} is dynamically determined using leader r_l's robot-state information set $\mathcal{S}_l(\tau_n)$. It is defined below in terms of the earliest arrival time T_{early} and the latest arrival time T_{late}.

$$T_{\text{wait}} = \left(1 + \beta_w \frac{n_{\text{req}}(q_c)}{n^{\max}_{\text{req}}}\right)(T_{\text{late}} - T_{\text{early}}) \tag{9}$$

where β_w is an adjustment coefficient. In this paper, β_w is set to 1.25 based on experiments. Here, $n_{\text{req}}(q_c)$ denotes the number of robots required for subtask q_c, and $n^{\max}_{\text{req}} = \max_{q_j \in \mathcal{Q}} n_{\text{req}}(q_j)$ denotes the maximum robot requirement across all subtasks. This design uses the arrival-time spread $(T_{\text{late}} - T_{\text{early}})$ as the baseline waiting window and scales it by $n_{\text{req}}(q_c)/n^{\max}_{\text{req}}$, so subtasks requiring more robots are given a proportionally longer waiting time to improve collaboration success while avoiding excessive delay for smaller teams. The rationale is that larger teams are harder to synchronize under limited communication: more robots imply greater arrival-time variability, so the probability that all required robots arrive within a short window is lower.

When leader r_l observes that the collaboration waiting time has reached T_{wait} and the number of arrived robots is still below the required cardinality for q_c, timeout-triggered recruitment is activated. The procedure is summarized in Algorithm 1. The leader first performs local recovery (lines 3–12). It scans the reachable robot set $\mathcal{R}_b$, which consists of robots currently discoverable by r_l through direct links or multi-hop relays, and extracts robots that are both available and type-compatible with the missing roles of q_c. Here, available robots are those that are not executing other subtasks, are not already committed to recruitment, and have not been recruited by other robots. This locally feasible set is denoted by $\mathcal{R}^l_{\text{fit}}$. If the union of $\mathcal{R}_{\text{arr}}$ and $\mathcal{R}^l_{\text{fit}}$ is sufficient to satisfy the requirement of q_c, the leader recruits the required robots from $\mathcal{R}^l_{\text{fit}}$. The recruitment message includes the subtask ID of q_c, the missing role counts and types, and the publication timestamp τ_n.

During recruitment, leader r_l broadcasts recruitment messages and follows a first-come-first-served strategy among received bids. Bidder $r_b \in \mathcal{R}^l_{\mathrm{fit}}$ applies according to message publication time. Upon receiving a bid from robot $r_b \in \mathcal{R}^l_{\mathrm{fit}}$, leader r_l sends a lock request $\mathrm{I}^l_{\mathrm{lock}} = \{q_c, \mathrm{ID}_b\}$ to r_b, where ID_b is the ID of r_b. The unlocked bidder r_b replies with $\mathrm{I}^b_{\mathrm{agree}} = \{q_c, \mathrm{ID}_b\}$ and becomes locked. After successful recruitment, r_l broadcasts $\mathrm{I}^l_{\mathrm{finish}} = \{q_c\}$, and the locked robots proceed to q_c. Here, recruitment is regarded as successful only when the required replacement robots have accepted the recruitment request and arrived at the target region within the corresponding waiting bound. Leader r_l continues broadcasting recruitment messages until the required robots are obtained or the recruitment exceeds T_{bid}. The collaboration waiting time T_{wait} is then updated according to the predicted arrival times of the recruited robots. If the updated waiting bound is still exceeded, the leader declares the subtask failed. The bidding timeout T_{bid} is defined below.

$$T_{\mathrm{bid}} = |\mathcal{R}| \, \frac{1}{f_{\mathrm{com}}} \, N_{\mathrm{round}} + t_{\mathrm{comp}}, \tag{10}$$

where $|\mathcal{R}|$ is the total number of robots, N_{round} is the number of recruiter–bidder message rounds required by the recruitment process, and t_{comp} is the computation time. In this paper, N_{round} is set to 5 and t_{comp} is set to 2.0 s.

If local recovery is insufficient, the leader expands the search to the free market (lines 13–20). It first constructs a backup candidate set $\mathcal{R}^f_{\mathrm{fit}}$ from robots in $\mathcal{R}_f$ whose capabilities match the remaining missing roles of q_c. If the union of the arrived robots $\mathcal{R}_{\mathrm{arr}}$, the locally reachable candidates $\mathcal{R}^l_{\mathrm{fit}}$, and the free-market candidates $\mathcal{R}^f_{\mathrm{fit}}$ is still insufficient, the subtask is immediately treated as failed. Otherwise, the leader selects a recruiter r_p from the arrived robots according to $r_p = \arg\min_{r \in \mathcal{R}_{\mathrm{arr}}} d(x_r(\tau_n), x_{\mathrm{free}})$, where $x_r(\tau_n)$ is the position of robot r recorded in $\mathcal{S}_l(\tau_n)$. The recruitment procedure executed by recruiter r_p in the free market is the same as the one used by leader r_l. Leader r_l declares subtask failure if recruitment is unsuccessful within T^p_{leave}. The leave timeout is defined below.

$$T^p_{\mathrm{leave}} = \left(1 + \zeta \frac{d_{\mathrm{leave}}}{d_{\max}}\right) \left(\frac{2 d_{\mathrm{leave}}}{v_p}\right) + T_{\mathrm{bid}} \tag{11}$$

where $d_{\max}$ is the maximum inter-target-region distance, and ζ is an adjustment coefficient. Based on experiments, ζ is set to 0.75 in this paper. d_{leave} denotes the distance from recruiter r_p to the free market. By adapting T^p_{leave} to travel distance, the leader avoids declaring failure too early for distant recruiters and avoids excessive waiting for nearby recruiters. The first term in Eq. (11) estimates the round-trip travel time between the subtask region and the free market, while T_{bid} reserves time for recruitment in the market.

If recruitment is still unsuccessful within T^p_{leave}, subtask q_c is declared failed, and the leader notifies all robots currently executing q_c. Failed subtasks are added to each robot's failed-subtask set and are temporarily skipped until robots enter the free market.

2) Failure Handling for Collaborative Subtasks: Failed collaborative subtasks are handled through free-market-based reassignment. Once a robot enters the free market, failed subtasks are reallocated through a recruiter–bidder mechanism after state information is synchronized, enabling robots to recover from collaboration failures using updated information.

When robot r_k finds that its current optional-subtask set contains only failed subtasks, it enters the free market for reassignment. After arriving at the free market, r_k first synchronizes recruitment messages and state information, and then checks whether it can serve as a recruiter. Let $\mathcal{F}_k$ denote the failed collaborative subtask set maintained by r_k. Let $\mathcal{R}_{\mathrm{rec}}$ denote the current recruiter set in the free market, and let $\mathcal{R}^<_k \subseteq \mathcal{R}_{\mathrm{rec}}$ denote the recruiters that published earlier than r_k. Robot r_k becomes a recruiter only if it has not received any valid recruitment message for a compatible subtask. In that case, it selects one failed subtask from $\mathcal{F}_k$ according to the shortfall-and-distance rule and publishes a recruitment message for that subtask.

Let $n^k_{\mathrm{rem}}(t, j)$ represent the remaining number of type-j robots that are available to recruiter r_k at time t after accounting for earlier recruitment requests. For each failed subtask $q \in \mathcal{F}_k$, the shortfall is defined below. $\Delta(q) = \sum_{j \in \mathcal{T}} \max\big(0,\ n_{\mathrm{req}}(q, j) - n^k_{\mathrm{rem}}(t, j)\big)$. Recruiter r_k then selects the failed subtask with the minimum shortfall. The selected subtask is defined below.

$$q^*_k = \arg\min_{q \in \mathcal{F}_k} \Delta(q). \tag{12}$$

If multiple candidates share the same minimum shortfall, r_k selects the one whose target region is nearest. After determining the subtask to be handled first, r_k publishes a recruitment message $\mathrm{I}^k_{\mathrm{bid}} = \{\tau_k, r_k, q^*_k, \eta_k\}$. Here, τ_k is the publishing time, and η_k is the type-cardinality requirement vector of the recruitment message.

After recruitment messages are published, any robot in the free market that is not acting as a recruiter is treated as a bidder. Bidder r_c selects a recruitment message according to task priority and message publishing time. Let $\mathcal{M}_c(\tau_n)$ be the set of active recruitment messages received by bidder r_c at time τ_n. The compatible message set is defined as $\mathcal{M}^{\mathrm{com}}_c(\tau_n) = \left\{\mathrm{I}^k_{\mathrm{bid}} \in \mathcal{M}_c(\tau_n) \;\middle|\; r_c \text{ is compatible with } q^*_k\right\}$. For each message $\mathrm{I}^k_{\mathrm{bid}}$, let $\rho(q^*_k)$ denote the priority rank of subtask q^*_k, where a smaller value means higher priority. The bidder-selected recruitment message is defined as $\mathrm{I}^*_c = \arg\min_{\mathrm{I}^k_{\mathrm{bid}} \in \mathcal{M}^{\mathrm{com}}_c(\tau_n)} \big(\rho(q^*_k), \tau_k\big)$, where minimization is lexicographic: bidders first select the highest-priority subtask, and among subtasks with the same priority, they choose the earliest published recruitment message. In this paper, high-priority subtasks refer to emergency subtasks issued by leader-dispatched recruiters, whereas low-priority subtasks refer to failed or currently unassigned subtasks handled through free-market reassignment.

During recruitment, recruiter r_k may receive a compatible recruitment message $\mathrm{I}^h_{\mathrm{bid}}$ and become a potential bidder. If the currently available free-market robots are insufficient to satisfy both recruitments simultaneously, r_k compares $(\rho(q^*_h), \tau_h)$ with $(\rho(q^*_k), \tau_k)$. If $\rho(q^*_h) < \rho(q^*_k)$, or if $\rho(q^*_h) = \rho(q^*_k)$ and $\tau_h < \tau_k$, then r_k relinquishes its recruiter role, cancels its own recruitment, and switches to bidder mode for $\mathrm{I}^h_{\mathrm{bid}}$.

Recruiter r_k follows the same first-come-first-served rule described in Sec. VI-C1. Let $\mathcal{L}_k$ denote the bidder set currently locked by recruiter r_k. Upon receiving a bid from robot r_c, recruiter r_k sends a lock request $\mathrm{I}^k_{\mathrm{lock}} = \{q^*_k, \mathrm{ID}_c\}$ to r_c, where q^*_k is the recruited subtask and ID_c is the ID of r_c. The uncommitted bidder r_c replies with $\mathrm{I}^c_{\mathrm{agree}} = \{q^*_k, \mathrm{ID}_c\}$ and becomes locked. Once recruitment is completed, r_k broadcasts $\mathrm{I}^k_{\mathrm{finish}} = \{q^*_k\}$, and locked robots then proceed to execute q^*_k.

A bidder is unlocked if it receives no message from recruiter r_k within T_{send} seconds, which indicates a presumed recruiter failure, or if a higher-priority subtask arrives. The unlocking timeout is defined below.

$$T_{\mathrm{send}} = \left(1 + \left\lceil \frac{d_{c,k}}{D} \right\rceil\right) \frac{1}{f_{\mathrm{com}}}, \tag{13}$$

where $d_{c,k}$ is the distance from bidder r_c to recruiter r_k, D is the communication range, and f_{com} is the communication frequency. In such cases, bidder r_c transmits $\mathrm{I}^c_{\mathrm{unlock}} = \{q^*_k, \mathrm{ID}_c\}$ to prompt r_k to remove it from the locked-robot list $\mathcal{L}_k$.

Lemma 3. *Under the assumptions in Sec.III-B, failed collaborative subtasks handled through free-market reassignment can obtain feasible reassignment plans under the assumptions in Sec.III-B.*

Proof. Consider a failed collaborative subtask q. For each type index $j \in \mathcal{T}$, let $n_{\mathrm{req}}(q, j)$ denote the required number of type-j robots, and let $n_{\mathrm{free}}(t', j)$ denote the number of type-j robots in the free market at time t'. Take any robot r_k capable of executing q.

Since q has already entered the collaborative-coordination stage before failure is declared, all prerequisite subtasks of q must have been completed. Thus, for any robot associated with q, the waiting time before either completing the current branch or switching to free-market recovery is finite. Let this worst-case waiting time be bounded by

$$T^{\max}_{\mathrm{timeout}} \le (1+\beta_w)\frac{d_{\max}}{v_k} + (1+\zeta)\frac{2d_{\max}}{\min_{r\in\mathcal{R}} v_r} + T_{\mathrm{bid}},$$

using the conservative bounds induced by Eqs. (9) and (11), where $d_{\max}$ is the maximum workspace distance. Let $T^{\max,k}_{\mathrm{task}}$ denote the maximum execution time of a subtask for r_k. Since each subtask execution takes finite time and $|\mathcal{Q}| < \infty$, robot r_k enters the free market within a finite time satisfying

$$T^k_{\mathrm{bound}} \le |\mathcal{Q}| \left(\frac{d_{\max}}{v_k} + T^{\max,k}_{\mathrm{task}} + T^{\max}_{\mathrm{timeout}} \right) < \infty.$$

Now consider the recruitment process associated with q. Since all robots capable of executing q enter the free market within finite time, the set of compatible recruitment messages is finite. Let $\mathcal{I}(q) = \{I_1, \dots, I_M\}, \qquad I_1 \prec \cdots \prec I_M$, where $\prec$ denotes publication order. A message is called *resolved* if it is either completed or interrupted and removed from the active set. By the bidder-selection rule, if $i < j$, then a bidder compatible with both I_i and I_j selects I_i; hence a later message cannot preempt an earlier compatible one. Therefore, by induction on the order index, every $I_m \in \mathcal{I}(q)$ is eventually resolved. If the recruitment associated with q has not yet been published, then all robots capable of executing q must be engaged in earlier compatible messages. Since the set of such earlier messages is finite and each of them is eventually resolved, the recruitment associated with q is eventually published. Finally, interruptions cannot occur infinitely often. Indeed, each interruption is caused by a higher-priority recruitment, and the number of higher-priority subtasks is at most $|\mathcal{Q}|$. Hence the recruitment associated with q can be interrupted only finitely many times. After the last interruption, its published message becomes the earliest active compatible one. By assumption iii), sufficiently many robots of each required type eventually become available, so this message is completed. Therefore, the recruitment associated with q is eventually completed, and q can be allocated. □

D. Event-Triggered Replanning Mechanism

The replanning mechanism described in this subsection is the CBTA-based update of local plans during online execution. It revises local task assignments when newly received information changes a robot's view of the current task state. This mechanism is distinct from the free-market reassignment process for failed collaborative subtasks. In particular, CBTA replanning is used only when a robot has a nonempty optional subtask set and is not currently engaged in collaborative-recovery procedures. These procedures include timeout-triggered recruitment for collaborative subtasks and recruiter–bidder-based reassignment in the free market.

During execution, each robot r_k maintains and periodically broadcasts a robot-state information set $\mathcal{S}_k(\tau_n)$, which records the information collected by r_k at time τ_n. In particular, each robot broadcasts peer states received from other robots, thereby enabling multi-hop information propagation. The robot-state information set is defined as $\mathcal{S}_k(\tau_n) = \{\mathcal{S}_{m|k}(\tau_n) \mid r_m \in \mathcal{R},\ m \neq k\}$. Each element $\mathcal{S}_{m|k}(\tau_n)$ is defined as follows: $\mathcal{S}_{m|k}(\tau_n) = \{\tau_m, q_m, a_m, t^m_a, t^m_s, x_m(\tau_n), \Pi_m(\tau_n)\}$. Here, $\mathcal{S}_{m|k}(\tau_n)$ records the latest status of robot r_m as received by r_k at time τ_n. Specifically, τ_m is the generation timestamp of the latest status message from r_m; q_m is the subtask currently committed by r_m; $a_m \in \mathrm{LA}(q_m)$ is the concrete action role undertaken by r_m for q_m; t^m_a is the estimated remaining travel time from the current estimated position of r_m to the target region of q_m; and t^m_s is the estimated task start time. Let $\mathcal{A}_m$ denote the set of estimated arrival times of other robots assigned to the same task region. Then t^m_s is defined as the non-negative difference between the estimated arrival time of r_m itself and the $n_{\mathrm{req}}(a_m)$-th smallest element in $\mathcal{A}_m$. In addition, $x_m(\tau_n)$ denotes the position of r_m at time τ_n, and $\Pi_m(\tau_n)$ denotes the plan of r_m at time τ_n.

To reduce unnecessary computation under limited communication, CBTA replanning is triggered only by events that update a robot's local task view or indicate changes in task-execution status. Specifically, these events include the reception of task-state information with newer timestamps, completion or failure of the robot's current subtask, task-allocation conflicts, and failure inference for non-collaborative subtasks, as illustrated in Fig.2.

1) Conflict-Triggered Replanning: During execution, limited communication may lead to task conflicts. To address this, a conflict-triggered replanning mechanism is introduced.

Each robot classifies subtasks into three categories at time τ_n: i) unassigned subtasks, whose currently committed robots are still insufficient in number or type, so additional robots may still join; ii) completed subtasks, whose required actions have already been executed and therefore no longer require allocation; and iii) confirmed subtasks, whose current commitments have been verified to satisfy all participation requirements and are thus ready for execution or are already under execution. Let $\mathcal{T}^k_{\text{not}}(\tau_n)$ and $\mathcal{T}^k_{\text{already}}(\tau_n)$ denote the unassigned-subtask set and completed-subtask set maintained by robot r_k, respectively. During communication, robots exchange these sets and merge newly received updates. In this way, the optional subtask set is maintained online and remains consistent with the latest distributed task state.

Based on the robot-state information set $\mathcal{S}_k(\tau_n)$, the unassigned-subtask set $\mathcal{T}^k_{\text{not}}(\tau_n)$, and the completed-subtask set $\mathcal{T}^k_{\text{already}}(\tau_n)$, robot r_k periodically checks for conflicts while executing $a_j \in \text{LA}(q_i)$. Two types of conflicts are considered. i) Conflict caused by task completion: At each check, r_k verifies whether q_i has entered $\mathcal{T}^k_{\text{already}}(\tau_n)$. If so, the current commitment is immediately invalidated, since further motion for q_i is redundant and may delay other pending tasks. Robot r_k then stops and replans. ii) Over-assignment: If the number of robots committed to a_j exceeds the required cardinality $n_{\text{req}}(a_j)$, robot r_k ranks all committed robots by their remaining travel time to the target region, with smaller t^m_a taking higher priority, and keeps only the first $n_{\text{req}}(a_j)$ robots as valid executors of a_j. If multiple robots have the same t^m_a, ties are broken by robot ID in ascending order. If r_k is not selected after this filtering step, it immediately stops the current commitment and replans.

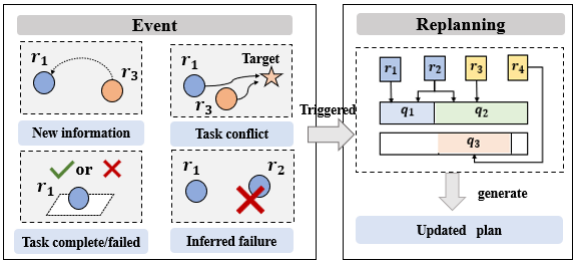

Fig. 2. Schematic illustration of the event-triggered replanning mechanism

2) Replanning for Non-Collaborative Subtasks: This replanning is applied only to non-collaborative subtasks, because collaborative subtasks are handled by a dedicated resilient task coordination mechanism. In this paper, failure of a non-collaborative subtask is considered only in the case where its committed robot suffers an irreversible failure and can no longer continue participating in the multi-robot system. Such a robot failure is modeled as a complete loss of both communication and action capabilities, and is therefore treated as the removal of that robot from the team. Under limited communication, other robots cannot directly distinguish such execution failure from delayed or incomplete information propagation. Therefore, each robot applies a timeout-based rule to assess whether the corresponding task-state record remains valid and, if necessary, triggers CBTA-based replanning when it has a nonempty optional subtask set and is not currently engaged in collaborative-recovery procedures.

Even for non-collaborative subtasks, delayed and incomplete information propagation may cause the same subtask to be assigned to multiple robots. Let $\mathcal{H}_k(q_m) = \{h \mid \mathcal{S}_{h|k}(\tau_n) \in \mathcal{S}_k(\tau_n),\ q_h = q_m\}$ denote the robot-index set induced by the state records in $\mathcal{S}_k(\tau_n)$, where q_h is the subtask component in $\mathcal{S}_{h|k}(\tau_n)$. For each $h \in \mathcal{H}_k(q_m)$, robot r_k computes a timeout candidate $T^{k,h}_{\text{late}}$ by Eq. (15) and uses the largest one as the expiry threshold for q_m. The expiry threshold is defined below.

$$T^k_{\text{late}}(q_m) = \max_{h \in \mathcal{H}_k(q_m)} T^{k,h}_{\text{late}}. \tag{14}$$

The timeout candidate is defined below.

$$\begin{aligned} T^{k,h}_{\text{late}} = \alpha_{\text{late}} \left(1 + \left\lceil \frac{d_{k,h}}{D} \right\rceil\right) \max\left(T_{\text{enc}}, \frac{1}{f_{\text{com}}}\right) \\ + t^h_s + \max_{a_i \in \text{LA}(q_m)} \text{LC}(a_i) \end{aligned} \tag{15}$$

$$T_{\text{enc}} = \frac{1}{4Dv_a\rho_r} \tag{16}$$

where α_{late} is a tuning parameter. In this paper, α_{late} is set to 2.5 based on experimental results. The value is chosen to be greater than 1 so that the practical information expiration time remains above its theoretical estimate. $d_{k,h}$ denotes the distance between robot r_k's current position and robot r_h's estimated position. The position of r_h is estimated based on its last known position x_h at the most recent communication time, together with its task plan. The estimation is carried out as follows: the current time is compared with the subtask start time in the plan to determine which task the robot is executing. If the current time exceeds the planned completion time of that task, the robot is considered to be in the free market. Otherwise, the robot is assumed to move at a constant speed along a straight line between task points, and its current position is estimated accordingly. T_{enc} denotes the average encounter time [40], where v_a is the average speed of robots and ρ_r is the robot density. Specifically, if current time τ_{now} satisfies $\tau_{\text{now}} - \tau_m > T^k_{\text{late}}(q_m)$, robot r_k treats the state record of robot r_m for subtask q_m as outdated. If robot r_k is capable of executing the corresponding action, q_m is non-collaborative, and $q_m \neq q_k$, then robot r_k triggers CBTA-based replanning, after which the updated local plan may take over q_m. Here, τ_m is the timestamp of the stored state record, and $T^k_{\text{late}}(q_m)$ is the expiry threshold adopted by robot r_k for subtask q_m.

E. Patrol Mechanism for Task Blocking

Temporal logic constraints can cause blocking while robots wait for prerequisite subtasks to be confirmed. For example, if subtask "inspect Zone B" is constrained to start only after "verify tunnel entrance" is completed, a robot assigned to Zone B may remain idle when the completion message from the entrance team cannot be received in time under limited communication. To address this issue, a patrol mechanism is proposed to proactively gather and relay prerequisite status information, thereby releasing blocked subtasks earlier. Let $\mathcal{Q}_k \subseteq \mathcal{Q}$ denote the set of subtasks executable by robot r_k. For each $q_i \in \mathcal{Q}_k$, let $\mathcal{P}_i$ denote the prerequisite-subtask

set of q_i. For each prerequisite subtask $q_j \in \mathcal{P}_i$, robot r_k maintains an estimated completion time $\hat{t}_k(q_j)$. The estimate $\hat{t}_k(q_j)$ is computed in the same form as $T^{k,m}_{\text{late}}$ in Sec. VI-D2. For subtasks constrained only by "do not start before the prerequisite starts", the same estimation logic is used, except that the prerequisite start time is estimated by subtracting the prerequisite execution duration from $\hat{t}_k(q_j)$.

A robot triggers the patrol mechanism only when two conditions are simultaneously satisfied: i) there is no currently unassigned optional subtask that can be executed immediately, and ii) for at least one pending subtask q_i, the current time exceeds the latest estimated completion time among its prerequisites, i.e., $\tau_{\text{now}} > \max_{q_j \in \mathcal{P}_i} \hat{t}_k(q_j)$. This means the robot has waited beyond the expected prerequisite completion horizon but still cannot confirm prerequisite satisfaction from local information. In this case, the robot starts patrol to actively collect missing status updates. The initial patrol targets include all regions associated with unresolved prerequisite subtasks and the free market. During patrol, the target set is refined as follows: i) remove regions whose prerequisites are already completed or reliably verified, ii) skip prerequisites whose successor subtasks have already started, iii) omit prerequisites whose own predecessor conditions are still unmet, and iv) update optional-subtask candidates and their estimated completion times using newly gathered information. Once at least one executable optional subtask is identified, the robot exits patrol and switches to task execution.

After patrol, if the estimated completion time is updated, the robot will patrol again based on the new timing estimates; otherwise, it will wait t_p seconds before the next patrol. The patrol interval is defined below.

$$t_p = \max_{q_i \in \mathcal{Q}_k} \max_{q_j \in \mathcal{P}_i} \left(\frac{d^j_f}{v_a} + \max_{a_\ell \in \text{LA}(q_j)} \text{LC}(a_\ell) \right) \tag{17}$$

where d^j_f is the free-market-to-subtask distance. t_p assumes the subtask has failed and will be reassigned in the free market.

F. Discussion

Theorem 1. *Under the assumptions in Sec.III-B, the proposed algorithm is complete. That is, if there exists a feasible plan satisfying φ, then the proposed algorithm is able to find a plan that satisfies φ.*

Proof. By Lemma 1, the pruning operation does not destroy completeness. Since $\mathcal{B}^-$ is a finite automaton, a depth-first search over $\mathcal{B}^-$ can find such a feasible accepting path whenever it exists. Based on the accepting path , the task decomposition step extracts the corresponding subtask sequence and the temporal dependencies among subtasks. These subtasks and dependencies are derived from an accepting path of $\mathcal{B}^-$, and therefore preserve the temporal-logic requirements encoded by φ. It remains to show that feasible robot assignments can be generated for the decomposed subtasks. For optional subtasks, CBTA evaluates feasible timetable insertions for type-compatible robots and resolves over-assignment conflicts by retaining only the required number of robots according to the prescribed ordering rule. Based on Lemma 2, whenever the currently available information contains enough type-compatible robots for an optional subtask, CBTA can generate a feasible plan. For a collaborative subtask recovered in the free market, by Lemma 3, a feasible reassignment can be found. Since the decomposed subtask set is finite, each subtask is eventually allocated either by CBTA with sufficient known candidates or by free-market-based reassignment when local information is insufficient. Hence, the algorithm can generate feasible local plans for all decomposed subtasks while respecting the extracted temporal dependencies. The induced subtask sequence is accepted by $\mathcal{B}^-$ and therefore satisfies the original sc-LTL specification φ. □

VII. RESULTS

The algorithm is evaluated through simulations and physical experiments. LTL formulas are converted to NBAs using LTL2BA [41]. Simulations are run in Python 3 on an 11th Gen Intel® Core™ i7-11390H with 16 GB RAM. Physical experiments are conducted on Ubuntu 18.04 with ROS.

A. Numerical Simulation

The simulation study evaluates the proposed framework under limited communication. The workspace contains 12 target regions, denoted by $\mathcal{L} = \{l_1, l_2, \ldots, l_{12}\}$, as shown in Fig. 3. To examine robustness with respect to task-location assignments, the region labels $l_1 \sim l_{12}$ are treated as permutable. Accordingly, 25 randomized tset instances (Inst. 1–Inst. 25) are generated by varying the label-to-location mapping while keeping the workspace geometry unchanged.

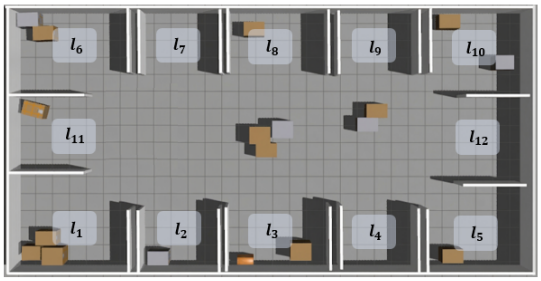

Fig. 3. Simulation environment for Tests 1–4. The simulation environment contains 12 target regions $l_1 \sim l_{12}$.

1) Parameter Setting: The parameters waiting-time scaling coefficient β_w, leave-time adjustment coefficient ζ , and expiration-time scaling coefficient α_{late}, which appear in Eqs.9, 11, and 15, respectively, are calibrated by one-at-a-time coordinate search. At each stage, one parameter is varied over its candidate set while the remaining parameters are fixed, and the value minimizing the average task completion time over all calibration runs is selected. Consider the LTL-specified global task $\varphi = \Diamond((l^{12}_{2,1} \wedge l^{12}_{1,1}) \wedge \Diamond((l^{7}_{2,2} \wedge \neg(l^{12}_{2,1} \wedge l^{12}_{1,1})) \wedge \Diamond(l^{6}_{2,1} \wedge \neg l^{7}_{2,2}))) \wedge \Diamond l^{2}_{3,1} \wedge \Diamond l^{3}_{3,1} \wedge \Diamond(l^{9}_{3,3} \wedge \neg p_4)$. After preprocessing φ, the resulting subtask set is $\mathcal{Q} = \{q_1, q_2, q_3, q_4, q_5, q_6\}$, where $q_1 = \{1, \{l^9_{3,3}\}, \{p_4\}\}$, $q_2 = \{2, \{l^3_{3,1}\}, \{\}\}$, $q_3 = \{3, \{l^2_{3,1}\}, \{\}\}$, $q_4 = \{4, \{l^{12}_{2,1}, l^{12}_{1,1}\}, \{\}\}$, $q_5 = \{5, \{l^7_{2,2}\}, \{\}\}$,$q_6 = \{6, \{l^6_{2,1}\}, \{\}\}$. The calibration set is evaluated over all 25 test instances under the 12-robot

setting and communication ranges $D \in \{3, 5, 10, 15, 20\}$. Since β_w and ζ mainly affect collaborative recovery, their calibration uses collaborative-failure realizations generated by removing one robot committed to a randomly selected collaborative subtask. The same failure realizations are reused for all candidate values. The candidate sets are $\beta_w \in \{0.75, 1.0, 1.25, 1.5, 1.75\}$, $\zeta \in \{0.5, 0.75, 1.0, 1.25, 1.5\}$, and $\alpha_{\text{late}} \in \{1.75, 2.0, 2.25, 2.5, 2.75\}$. The calibration results are shown in Fig. 4. Based on these results, we set $\beta_w = 1.25$, $\zeta = 0.75$, and $\alpha_{late} = 2.5$. The selected parameters are fixed for all subsequent tests.

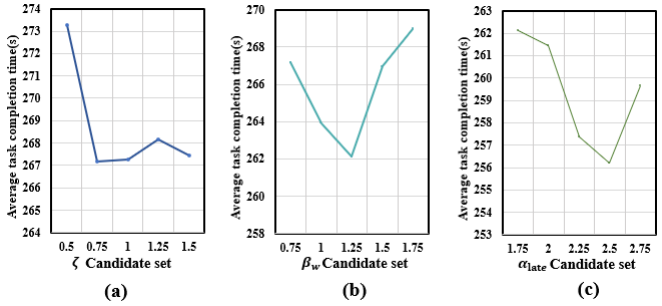

Fig. 4. Parameter-setting results.

After parameter calibration, four tests are conducted to evaluate different aspects of the proposed framework. Test 1 evaluates the performance of the proposed framework under dynamic changes, including online robot addition and robot removal. Test 2 evaluates the patrol mechanism through an ablation study. Test 3 compares timeout-triggered recruitment with timeout-based local recruitment [36] to examine whether expanding the replacement search through the free market improves collaborative recovery. Test 4 evaluates free-market-based reassignment for failed collaborative subtasks and compares it with failure-to-tail reassignment under severe communication constraints. Together, these tests provide an overall evaluation of the proposed framework from different perspectives under limited communication.

2) Test 1: Considering the same global task φ as in Parameter Setting, Test 1 evaluates the overall performance of the proposed framework under dynamic team changes. Specifically, the initial team size is selected from $|\mathcal{R}| \in \{8, 12, 15\}$, corresponding to type compositions $(2, 2, 4)$, $(4, 4, 4)$, and $(5, 5, 5)$ for type-1, type-2, and type-3 robots, respectively, and the communication range is selected from $D \in \{3, 5, 10, 15, 20\}$. For each $(|\mathcal{R}|, D)$ setting, the evaluation is conducted over the 25 test instances. In each run, robot-addition and robot-removal events are introduced during execution to test whether the proposed framework can sustain task progress under dynamic team composition. Specifically, before subtask q_2 is executed, one additional type-3 robot is inserted near the target region of q_2. Later, during the coordination stage of collaborative subtask q_1, one robot already committed to q_1 is removed, leaving the subtask understaffed. Three metrics are used. The first is the *task completion time*, defined as the elapsed time from the start of execution to the completion of all required subtasks. The second is the *robot-addition reaction time*, defined as the elapsed time from the robot-addition event to the first time when the newly added robot's state information is received and used by at least one existing robot in the distributed allocation process. The third is the *collaborative-subtask recovery time*, defined as the elapsed time from robot removal to the time when the affected collaborative subtask is completed.

For each $(|\mathcal{R}|, D)$ setting, the reported results are obtained by averaging the above three metrics over the 25 test instances. The results are shown in Fig.5.Overall, performance degrades as the initial team size decreases and the communication range becomes smaller. This is because a sparser communication network slows state synchronization and makes collaboration more difficult. Under the 15-robot setting with $D = 20$ to 10, the newly added robot can quickly join the team coordination process, and the leader can recruit a qualified replacement locally after robot removal. As a result, the collaborative recovery time and the overall task completion time are both the smallest. However, when D is further reduced to 5 and 3, the degradation becomes much more pronounced, because robots must spend additional time moving to contact other robots or to recruit the robots required for collaborative subtasks. The cases with 12 and 8 robots follow a similar trend, except that the performance becomes worse as the team size decreases.

To further illustrate the dynamic execution process, we present a representative run with 12 robots and communication range $D = 10\,\text{m}$. As shown in Fig. 6, at $t = 15$, a new type-3 robot, r_{13}, joins the system and is assigned to subtask q_2 after entering communication range and synchronizing with the team. Later, at $t = 80$, robot r_{10} is removed, leaving collaborative subtask q_1 understaffed. The leader robot r_{11} first checks for a local replacement while keeping the remaining participants waiting. Once the waiting time exceeds T_{wait}, r_{11} activates timeout-triggered recruitment and dispatches r_9 as a recruiter. After recruitment succeeds, the team is re-formed, and q_1 is eventually completed by r_{11}, r_{13}, and r_9.

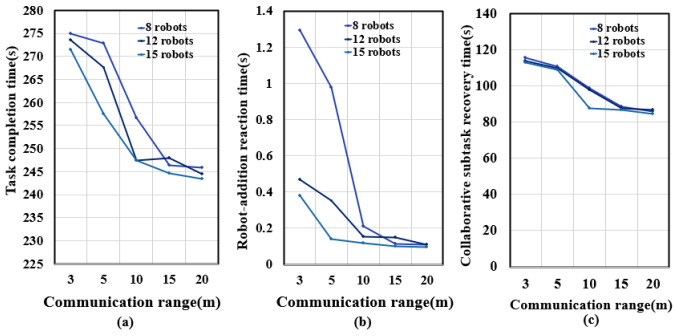

Fig. 5. Performance under dynamic team changes. (a) Task completion time. (b) Robot-addition reaction time. (c) Collaborative-subtask recovery time.

3) Test 2: Test 2 uses the same task and test instances as Test 1 to isolate the effect of the patrol mechanism. Two settings are compared: Case 1 enables patrol, whereas Case 2 disables patrol. All other robot settings, task instances, and communication ranges are kept identical.

As shown in Fig. 7, Case 2 exhibits a significant decline in task completion rate as D decreases, because blocked robots cannot reliably obtain prerequisite updates. Although some

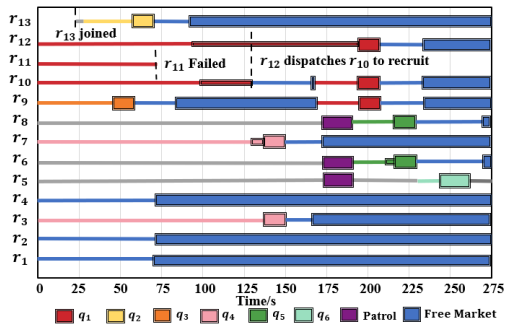

Fig. 6. Task timelines in Test 1. Wide blocks denote subtasks, narrow colored blocks indicate waiting. Gray lines mean no subtask and not in the free market, and other colored lines represent traveling toward the target region.

instances are still completed in Case 2 through incidental encounters, this recovery path is unstable and depends on geometry and trajectories. With patrol enabled, robots actively acquire missing prerequisite information, maintaining a 100% completion rate across all tested ranges. These results highlight the key advantage of the patrol mechanism: it provides robust information recovery rather than probabilistic recovery determined by incidental robot meetings.

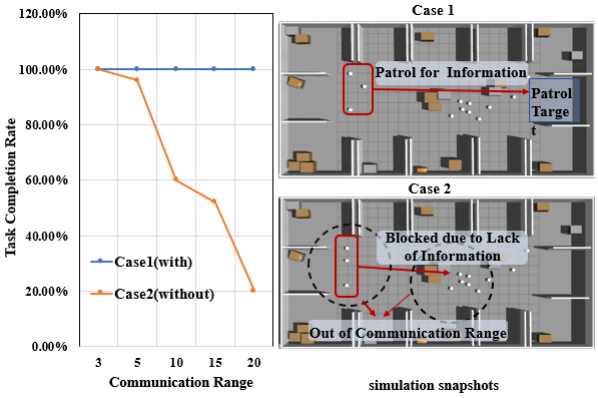

Fig. 7. Ablation study of the patrol mechanism in Test 2. Left: task completion rate under different communication ranges, where Case 1 enables patrol and Case 2 disables patrol. Right: representative snapshots showing that patrol actively collects missing prerequisite information, whereas robots without patrol may remain blocked due to the lack of communication.

4) Test 3: Considering the same global task φ as in Test 1, we further evaluate timeout-triggered recruitment. Timeout-triggered recruitment (TTR) is compared with timeout-based local recruitment (TLR) [36]. The two methods differ only in how replacement candidates are searched after timeout: TLR selects from robots currently reachable by the leader (usually the nearest one), while TTR can dispatch a recruiter to the free market to obtain additional qualified robots outside the local communication neighborhood. In each run, one robot is randomly removed during execution of q_1 to create the same failure trigger for both methods. Performance is measured by the completion time of type-3-related subtasks (T3-subtask time) under different communication ranges. We use this metric instead of total mission time because it directly captures recovery efficiency for the affected collaborative chain, while total mission time can be perturbed by unrelated subtasks.

Fig. 8 shows that TTR yields shorter T3-subtask time in most instances, with the largest gains at small communication ranges. The reason is that, under sparse connectivity, TLR is constrained by local visibility and may not find a feasible replacement quickly, whereas TTR expands the search scope through the free market and therefore restores collaboration earlier. When communication range is large, the free market is effectively inside the connected neighborhood, so TTR and TLR have similar candidate sets and similar performance. The exceptions (e.g., Inst. 2 and Inst. 6) are also explainable: both methods behave similarly because q_2 and q_3 are far from the initial robot positions and no suitable type-3 robot is available in the free market at timeout; here the dominant bottleneck is temporary resource scarcity, not the recruitment strategy. Overall, the experiment indicates that TTR improves robustness mainly by enlarging the effective replacement-search region under communication constraints.

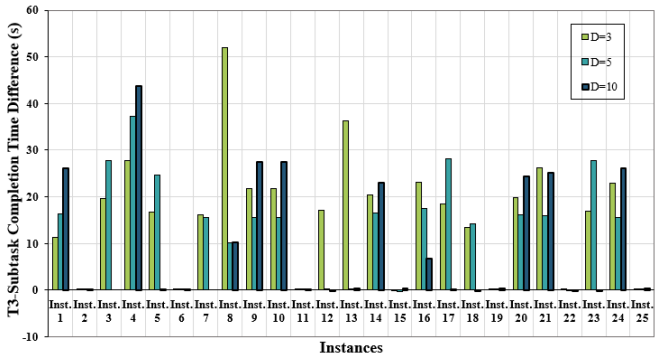

Fig. 8. Comparison between timeout-triggered recruitment (TTR) and timeout-based local recruitment (TLR). The vertical axis reports $T_{\mathrm{TLR}} - T_{\mathrm{TTR}}$ for type-3-related subtasks; positive values indicate that TTR achieves shorter recovery-related completion time.

5) Test 4: In scenarios with multiple collaborative subtasks under limited communication, repeated failures can propagate across temporally dependent tasks and substantially delay mission completion. This test evaluates whether free-market-based reassignment (FMR) can reduce such cascading delays compared with failure-to-tail reassignment (F2TR), where each failed subtask is appended to the end of the local plan and retried later. The comparison is conducted over Inst. 1–Inst. 25 under $D = 1.5\,\mathrm{m}$, using task completion time as the primary metric. Consider the LTL-specified task $\varphi_1 = \Diamond l^{11}_{1,2} \wedge \Diamond l^{7}_{1,2} \wedge \Diamond l^{2}_{1,2} \wedge \Diamond((l^{2}_{2,1} \wedge \neg l^{9}_{2,2}) \wedge \Diamond l^{9}_{2,2})$. Let $\mathcal{Q} = \{q_1, q_2, q_3, q_4, q_5\}$, where $q_1 = \{1, \{l^{2}_{2,1}\}\}$, $q_2 = \{2, \{l^{9}_{2,2}\}\}$, $q_3 = \{3, \{l^{2}_{1,2}\}\}$, $q_4 = \{4, \{l^{7}_{1,2}\}\}$, and $q_5 = \{5, \{l^{11}_{1,2}\}\}$. Five robots are deployed: r_1, r_2, r_3 (type 1) and r_4, r_5 (type 2). To trigger representative collaborative failures, one type-1 robot is removed before the second (in execution order) type-1 collaborative subtask is completed. This setting is critical because the team has minimal redundancy: a single removal can break the feasibility of the current collaboration and force reallocation. Under F2TR, this often leads to deferral and repeated waiting in downstream subtasks. In contrast, FMR synchronizes failure information in the free market and reallocates with a broader and more up-to-date candidate set,

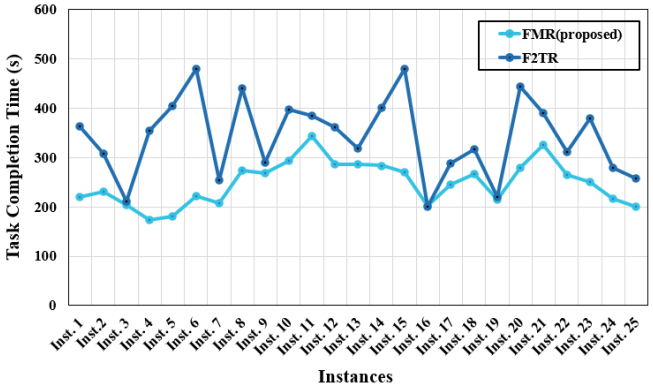

Fig. 9. Comparison between FMR (proposed) and F2TR.

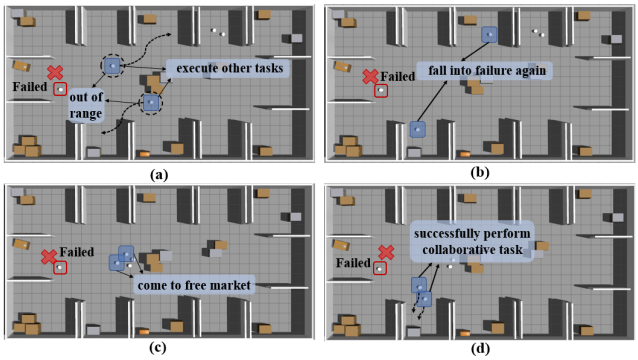

Fig. 10. Schematic comparison of collaborative-failure handling processes in F2TR and the proposed method. Subfigures (a) and (b) illustrate the F2TR process, while (c) and (d) illustrate the process of the proposed method.

which is expected to reduce repeated failures and shorten recovery latency.

As shown in Fig. 9, FMR generally outperforms F2TR across most instances, indicating that the benefit is systematic rather than case-specific. As shown in Fig. 10, the key reason is that, under dense collaborative dependencies and limited communication, a single failure often creates a "failure–delay–reassignment" loop: delayed recovery of one subtask postpones its successors, which increases the probability of additional waiting and repeated failures. F2TR tends to amplify this loop because failed subtasks are deferred to the tail and revisited with fragmented local information. In contrast, FMR interrupts the loop by synchronizing failure states in the free market and reallocating from a broader, more up-to-date candidate set, which shortens recovery latency and reduces repeated failure. The smaller gaps in instances such as Inst.3 and Inst.16 are also informative: when spatial layout and timing naturally permit sufficient information exchange, F2TR can approach the performance of FMR. This suggests that FMR's main advantage appears in communication-stressed or coordination-dense cases.

B. Physical Experiment

Consider the environment depicted in Fig. 11 (a), where l_1 denotes the sorting area, l_2 the storage center, and l_3 the monitoring room. Four robots of three distinct types

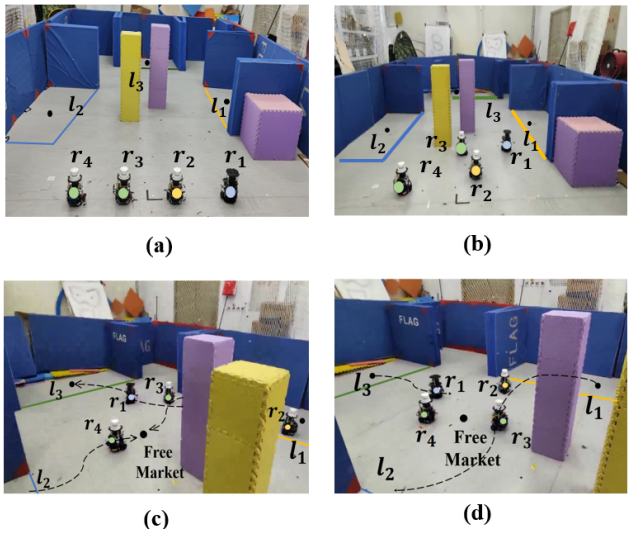

Fig. 11. The dashed lines indicate the robot trajectories. In (a), l_1 to l_3 represent the areas of interest. There are three robot types: $\{r_1\}$, $\{r_2\}$, and $\{r_3, r_4\}$. In (b)–(d), the robots are executing their subtasks.

operate with a 2.0-meter communication range. The mission specification $\varphi = \Diamond(l^1_{1,1} \wedge l^1_{3,1} \wedge l^1_{2,1}) \wedge \Diamond l^2_{3,1} \wedge \Diamond l^3_{1,1} \wedge \Diamond l^2_{2,1} \wedge (\neg l^2_{2,1} \mathcal{U}\, l^4_{1,1}) \wedge (\neg l^3_{1,1} \mathcal{U}\, (l^1_{1,1} \wedge l^1_{3,1} \wedge l^1_{2,1}))$ requires one robot of each type (1, 2, and 3) to inspect, classify, and process goods in the sorting area l_1; a type-3 robot to inspect the storage area l_2; and, upon completion of the sorting tasks, a type-1 robot to report in the monitoring room l_3, after which a type-2 robot processes goods in the storage area l_2.

Experimental results (Fig. 11) provides a feasibility demonstration of the proposed framework in a real communication-limited setting. First, the collaborative operation in l_1 is completed before the dependent actions in l_3, which is consistent with the temporal-order constraints in φ and indicates that prerequisite information is propagated in time to avoid logical blocking. Second, after leaving l_1, robot r_1 proceeds to l_3 while r_2 is temporarily outside direct 2.0-meter communication range; nevertheless, team coordination is preserved through free-market-mediated information exchange, so execution does not stall due to link interruption. Third, once the l_3-related subtask is confirmed complete, r_2 transitions to l_2 to execute the subsequent dependent task, showing that task handoff remains consistent even across intermittent connectivity. Overall, the physical experiment serves as a feasibility demonstration of the proposed execution framework under real range-limited communication.

VIII. CONCLUSIONS

This paper presents a distributed task allocation framework with resilient task coordination for heterogeneous multi-robot systems under temporal logic and communication constraints. Starting from a global sc-LTL specification, the framework derives executable subtasks through automaton pruning and task decomposition and generates initial plans using CBTA. To sustain task progress under limited communication, it combines a patrol mechanism for resolving task blocking with a resilient coordination mechanism for collaborative subtasks.

Simulations on 25 test instances and a physical experiment demonstrate improved robustness under incomplete and delayed team-state information.

REFERENCES

[1] Y. Qu, B. Xin, Q. Wang, R. Li, and Z. Du, "A novel memetic algorithm for distributed shape formation of swarm robots with both acceleration and velocity constraints," *Science China Information Sciences*, vol. 67, no. 12, p. 222201, 2024.

[2] H. Fang, S. Wei, Y. Shi, and J. Ou, "Multi-agent ergodic surveillance under unknown target distribution," *Unmanned Systems*, 2025, doi: 10.1142/S2301385025440054.

[3] H. Huang, W. He, and Z. Zeng, "An integral-enhanced adaptive gradient neural network for kwta and multirobot coordination," *IEEE Transactions on Cybernetics*, vol. 56, no. 6, pp. 3165–3176, 2026.

[4] L. Zhou, M. Jing, L. Zhong, Z. Yu, and X. Zeng, "Task-binding based branch-and-bound algorithm for noc mapping," in *2012 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2012, pp. 648–651.

[5] T. Okubo and M. Takahashi, "Simultaneous optimization of task allocation and path planning using mixed-integer programming for time and capacity constrained multi-agent pickup and delivery," in *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*. IEEE, 2022, pp. 1088–1093.

[6] Z. Liu, Z. Li, and M. Guo, "Collaborative planning of formal tasks based on action chains," *Scientia Sinica Informationis*, vol. 54, pp. 2623–2641, 2024, in Chinese.

[7] S. Wang, Y. Liu, Y. Qiu, and J. Zhou, "Consensus-based decentralized task allocation for multi-agent systems and simultaneous multi-agent tasks," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 12 593–12 600, 2022.

[8] "A decentralized control and task allocation framework for heterogeneous redundant multiagent systems."

[9] Y. Wang, S. Ren, and C. Wang, "Multi-UAV multi-task allocation based on cognitive state-based hybrid contract net protocol," in *2025 10th International Conference on Computer and Communication System (ICCCS)*. IEEE, 2025, pp. 921–926.

[10] A. Maoudj, B. Bouzouia, A. Hentout, A. Kouider, and R. Toumi, "Distributed multi-agent approach based on priority rules and genetic algorithm for tasks scheduling in multi-robot cells," in *IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 2016, pp. 692–697.

[11] J. Xie and C.-C. Liu, "Multi-agent systems and their applications," *Journal of International Council on Electrical Engineering*, vol. 7, no. 1, pp. 188–197, 2017.

[12] K. Leahy, D. Zhou, C.-I. Vasile, K. Oikonomopoulos, M. Schwager, and C. Belta, "Persistent surveillance for unmanned aerial vehicles subject to charging and temporal logic constraints," *Autonomous Robots*, vol. 40, no. 8, pp. 1363–1378, 2016.

[13] S. Xu, X. Luo, Y. Huang, L. Leng, R. Liu, and C. Liu, "NL2HLTL2PLAN: Scaling up natural language understanding for multi-robots through hierarchical temporal logic task specifications," *IEEE Robotics and Automation Letters*, pp. 1–8, 2025.

[14] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT Press, 2008.

[15] L. Li, Z. Chen, H. Wang, and Z. Kan, "Task allocation of heterogeneous robots under temporal logic specifications with inter-task constraints and variable capabilities," *IEEE Transactions on Automation Science and Engineering*, 2025.

[16] L. Li, Z. Zhou, Z. Chen, H. Wang, Z. Kan, and J. Qin, "A formal framework for reactive heterogeneous multirobot task allocation in uncertain semantic environments," *IEEE Transactions on Cybernetics*, pp. 1–14, 2026.

[17] P. Yu and D. V. Dimarogonas, "Distributed motion coordination for multirobot systems under LTL specifications," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 1047–1062, 2021.

[18] S. L. Smith, J. Tůmová, C. Belta, and D. Rus, "Optimal path planning under temporal logic constraints," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2010, pp. 3288–3293.

[19] A. Ulusoy, S. L. Smith, X. C. Ding, C. Belta, and D. Rus, "Optimality and robustness in multi-robot path planning with temporal logic constraints," *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 889–911, 2013.

[20] Y. Kantaros and M. M. Zavlanos, "Sampling-based optimal control synthesis for multirobot systems under global temporal tasks," *IEEE Transactions on Automatic Control*, vol. 64, no. 5, pp. 1916–1931, 2018.

[21] ——, "Stylus*: A temporal logic optimal control synthesis algorithm for large-scale multi-robot systems," *The International Journal of Robotics Research*, vol. 39, no. 7, pp. 812–836, 2020.

[22] X. Luo and M. M. Zavlanos, "Temporal logic task allocation in heterogeneous multirobot systems," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3602–3621, 2022.

[23] Z. Chen and Z. Kan, "Real-time reactive task allocation and planning of large heterogeneous multi-robot systems with temporal logic specifications," *The International Journal of Robotics Research*, vol. 44, no. 4, pp. 640–664, 2024.

[24] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Decomposition of finite LTL specifications for efficient multi-agent planning," in *Distributed Autonomous Robotic Systems: The 13th International Symposium*. Springer, 2018, pp. 253–267.

[25] ——, "Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems," *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 818–838, 2018.

[26] J. Liu, F. Li, X. Jin, and Y. Tang, "Distributed task allocation for multi-agent systems: A submodular optimization approach," *arXiv preprint arXiv:2412.02146*, 2024.

[27] Z. Liu, M. Guo, W. Bao, and Z. Li, "Fast and adaptive multi-agent planning under collaborative temporal logic tasks via poset products," *Research*, vol. 7, p. 0337, 2024.

[28] Y. Zhu *et al.*, "DEXTER-LLM: Dynamic and explainable coordination of multi-robot systems in unknown environments via large language models," *arXiv preprint arXiv:2508.14387*, 2025.

[29] I. Filippidis, D. V. Dimarogonas, and K. J. Kyriakopoulos, "Decentralized multi-agent control from local LTL specifications," in *2012 IEEE 51st IEEE Conference on Decision and Control (CDC)*, 2012, pp. 6235–6240.

[30] M. Guo and D. V. Dimarogonas, "Multi-agent plan reconfiguration under local LTL specifications," *The International Journal of Robotics Research*, vol. 34, no. 2, pp. 218–235, 2015.

[31] ——, "Distributed plan reconfiguration via knowledge transfer in multi-agent systems under local LTL specifications," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 4304–4309.

[32] D. Tian, H. Fang, Q. Yang, and Y. Wei, "Decentralized motion planning for multiagent collaboration under coupled LTL task specifications," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 6, pp. 3602–3611, 2021.

[33] Z. Chen, L. Li, and Z. Kan, "Distributed task allocation and planning under temporal logic and communication constraints," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6536–6543, 2024.

[34] C. H. Fua and S. S. Ge, "Cobos: Cooperative backoff adaptive scheme for multirobot task allocation," *IEEE Transactions on Robotics*, vol. 21, no. 6, pp. 1168–1178, 2005.

[35] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, and J. Bohg, "Dynamic multi-robot task allocation under uncertainty and temporal constraints," *Autonomous Robots*, vol. 46, no. 1, pp. 231–247, 2022.

[36] V. C. Kalempa, L. Piardi, M. Limeira, and A. S. de Oliveira, "Multi-robot preemptive task scheduling with fault recovery: A novel approach to automatic logistics of smart factories," *Sensors*, vol. 21, no. 19, p. 6536, 2021.

[37] C. Belta, B. Yordanov, and E. A. Gol, *Formal methods for discrete-time dynamical systems*. Springer, 2017, vol. 89.

[38] X. Luo, S. Xu, R. Liu, and C. Liu, "Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications," *IEEE Robotics and Automation Letters*, 2024.

[39] Z. Liu, M. Guo, and Z. Li, "Time minimization and online synchronization for multi-agent systems under collaborative temporal logic tasks," *Automatica*, vol. 159, p. 111377, 2024.

[40] C. Bettstetter, H. Hartenstein, and X. Pérez-Costa, "Stochastic properties of the random waypoint mobility model," *Wireless Networks*, vol. 10, no. 5, pp. 555–567, 2004.

[41] P. Gastin and D. Oddoux, "Fast LTL to Büchi automata translation," in *Computer Aided Verification*, G. Berry, H. Comon, and A. Finkel, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 53–65.